Rising Waters Flood Prediction System

Professional Project Documentation

Machine Learning-based Flood Prediction Web Application

Project Type	Flask Web Application with Machine Learning Pipeline
Final Model Artifact	models/floods.save
Scaler Artifact	models/transform.save
Dataset	data/flood_dataset.xlsx
Notebook	notebooks/FloodPrediction.ipynb
Training Script	src/train_models.py

Table of Contents

Project Overview

Rising Waters Flood Prediction System is a Flask-based web application that uses a trained XGBoost classification model to predict whether submitted weather and rainfall conditions indicate flood risk. The project includes an exploratory notebook, a preprocessing and training workflow, persisted model artifacts, a Flask backend, and responsive HTML/CSS/JavaScript frontend pages.

The application reads user inputs from a prediction form, converts them into a pandas DataFrame with the exact feature names used during training, applies the saved StandardScaler, and uses the saved model artifact to render either a flood-risk result page or a no-risk result page.

Problem Statement

Flood prediction is important for preparedness and risk awareness. This project addresses the problem of classifying flood risk from available weather and rainfall parameters such as temperature, humidity, cloud cover, annual rainfall, seasonal rainfall values, average June rainfall, and subdivision value.

The implementation focuses on a supervised binary classification task where the target column is `flood`, with `1` representing detected flood risk and `0` representing no detected flood risk.

Objectives

- Load and analyze the flood dataset from an Excel file.
- Perform exploratory data analysis using pandas, Matplotlib, and Seaborn.
- Check missing values and handle numerical outliers using IQR-based capping.
- Prepare input features and target labels for machine learning.
- Apply StandardScaler for feature scaling.
- Train and evaluate Decision Tree, Random Forest, KNN, and XGBoost classifiers.
- Compare models by accuracy and persist the best-performing model.
- Deploy prediction functionality through a Flask web application.
- Provide a responsive frontend for user input and prediction results.

Technology Stack

Category	Technologies Used
Programming Language	Python
Backend	Flask
Data Handling	Pandas, NumPy, OpenPyXL
Visualization	Matplotlib, Seaborn
Machine Learning	Scikit-learn, XGBoost
Model Persistence	Joblib
Frontend	HTML5, CSS3, Vanilla JavaScript
Deployment	Render, Gunicorn

Project Folder Structure

```
rising-waters-flood-prediction/
|-- app.py
|-- requirements.txt
|-- README.md
|-- data/
|   `-- flood_dataset.xlsx
|-- models/
|   |-- floods.save
|   `-- transform.save
|-- notebooks/
|   `-- FloodPrediction.ipynb
|-- src/
|   `-- train_models.py
|-- templates/
|   |-- home.html
|   |-- index.html
|   |-- chance.html
|   `-- no_chance.html
|-- static/
|   |-- css/
|   |   `-- main.css
|   `-- js/
|       `-- main.js
`-- screenshots/
    |-- home_page.png
    |-- prediction_form.png
    |-- flood_result.png
    `-- no_flood_result.png
```

Dataset Information

The project uses the Excel dataset located at `data/flood_dataset.xlsx`. The dataset contains 115 records and 11 columns. There are 10 input features and one target column named `flood`. The current dataset has no missing values.

Property	Value
Dataset path	data/flood_dataset.xlsx
Rows	115
Columns	11
Input features	10
Target column	flood
Missing values	0 total missing values
Target distribution	0: 99 records, 1: 16 records

Feature Columns

Feature	Data Type	Purpose in Application
Temp	Integer	Temperature input
Humidity	Integer	Humidity input
Cloud Cover	Integer	Cloud cover input
ANNUAL	Float	Annual rainfall value
Jan-Feb	Float	January-February rainfall value
Mar-May	Float	March-May rainfall value
Jun-Sep	Float	June-September rainfall value
Oct-Dec	Float	October-December rainfall value

Feature	Data Type	Purpose in Application
avgjune	Float	Average June rainfall value
sub	Float	Subdivision value
flood	Integer	Target variable

Exploratory Data Analysis (EDA)

EDA is implemented in `notebooks/FloodPrediction.ipynb`. The notebook contains 38 cells and is organized into data collection, visualization and analysis, and preprocessing sections.

- Dataset exploration: `head()`, `shape`, `columns`, `info()`, `describe()`, `dtypes`, and `isnull().sum()`.

- Descriptive statistical analysis: `describe()`, data types, unique value counts, missing values summary, and printed dataset summary.

- Univariate analysis: numerical feature detection, flood target exclusion, histogram with KDE, and boxplot generation using loops.

- Multivariate analysis: numerical correlation matrix, correlation heatmap, features correlated with `flood`, and scatter plots for ANNUAL, Jun-Sep, and Humidity against flood when columns exist.

Data Preprocessing

Preprocessing is implemented in the notebook before feature scaling and model preparation. The workflow checks missing values with `isnull().sum()` and `isnull().any()`. The current dataset contains no missing values, so no imputation logic is required by the current data.

Outliers are handled using the Interquartile Range method. For each numerical feature except the target column `flood`, lower and upper bounds are calculated using Q1, Q3, and IQR. Values outside the bounds are capped with `clip()` rather than removing rows.

Step	Implemented Behavior
Missing value check	Counts missing values and prints whether missing values exist.
Outlier detection	Uses IQR for each numerical feature except `flood`.
Outlier handling	Caps values using `clip()`; rows are not removed.
Categorical handling	Detects categorical columns; applies LabelEncoder only if such columns exist.

Feature Scaling

The project uses `StandardScaler` from Scikit-learn. In the notebook, the scaler is fitted on `X_train`, used to transform `X_train` and `X_test`, and saved as `models/transform.save` using Joblib. The Flask application loads this scaler once during startup and applies it to user form inputs before prediction.

The saved scaler artifact was verified to contain a Scikit-learn `StandardScaler`

Model Training

Model training is implemented in `src/train_models.py`. The script loads the dataset, loads the saved scaler, splits the dataset into features and target, performs `train_test_split` with `test_size=0.20` and `random_state=42`, scales the training and testing feature matrices, and trains reusable model functions.

.

Function	Model
decision_tree()	DecisionTreeClassifier(random_state=42)
random_forest()	RandomForestClassifier(random_state=42)
knn()	KNeighborsClassifier()
xgboost_model()	XGBClassifier(random_state=42, eval_metric='logloss')

Each model function trains the model, predicts on `X_test`, prints accuracy, confusion matrix, and classification report, and returns `(model, accuracy)`.

Model Comparison

The `compare_models()` function trains Decision Tree, Random Forest, KNN, and XGBoost, stores their accuracies, prints a formatted comparison table, identifies the best-performing model by accuracy, and saves only the selected model to `models/floods.save`.

If multiple models have the same highest accuracy, the implemented tie-break rule selects XGBoost as the final deployment model.

Final Model Selection (XGBoost)

The final saved deployment artifact is `models/floods.save`. The artifact content includes `XGBClassifier`, confirming that the deployed model file is an XGBoost classifier. The training script also explicitly prefers XGBoost when tied for the highest accuracy.

The repository does not store a separate static metrics report with exact final accuracy values; metrics are printed when `src/train_models.py` is executed.

Model Persistence (transform.save and floods.save)

Artifact	Path	Purpose
Scaler	models/transform.save	Stores the fitted StandardScaler used to transform prediction inputs.
Model	models/floods.save	Stores the final XGBoost classifier used by the Flask application.

Both artifacts are loaded once during Flask application startup in `app.py`. If loading fails, the prediction route returns the form page with an error message and HTTP status code 400.

Function	Model
decision_tree()	DecisionTreeClassifier(random_state=42)
random_forest()	RandomForestClassifier(random_state=42)
knn()	KNeighborsClassifier()
xgboost_model()	XGBClassifier(random_state=42, eval_metric='logloss')

Each model function trains the model, predicts on `X_test`, prints accuracy, confusion matrix, and classification report, and returns `(model, accuracy)`.

Function	Model
decision_tree()	DecisionTreeClassifier(random_state=42)
random_forest()	RandomForestClassifier(random_state=42)
knn()	KNeighborsClassifier()
xgboost_model()	XGBClassifier(random_state=42, eval_metric='logloss')

Each model function trains the model, predicts on `X_test`, prints accuracy, confusion matrix, and classification report, and returns `(model, accuracy)`.

Model Comparison

The `compare_models()` function trains Decision Tree, Random Forest, KNN, and XGBoost, stores their accuracies, prints a formatted comparison table, identifies the best-performing model by accuracy, and saves only the selected model to `models/floods.save`.

If multiple models have the same highest accuracy, the implemented tie-break rule selects XGBoost as the final deployment model.

Final Model Selection (XGBoost)

The final saved deployment artifact is `models/floods.save`. The artifact content includes `XGBClassifier`, confirming that the deployed model file is an XGBoost classifier. The training script also explicitly prefers XGBoost when tied for the highest accuracy.

The repository does not store a separate static metrics report with exact final accuracy values; metrics are printed when `src/train_models.py` is executed.

Model Persistence (transform.save and floods.save)

Artifact	Path	Purpose
Scaler	models/transform.save	Stores the fitted StandardScaler used to transform prediction inputs.
Model	models/floods.save	Stores the final XGBoost classifier used by the Flask application.

Both artifacts are loaded once during Flask application startup in `app.py`. If loading fails, the prediction route returns the form page with an error message and HTTP status code 400.

Flask Application Workflow

The Flask backend is implemented in `app.py`. It initializes the Flask app, defines model and scaler paths, loads artifacts at startup, defines exact feature names, and exposes the user-facing routes.

Route	Method	Behavior
/	GET	Renders `home.html`.
/predict	GET	Renders `index.html`, the prediction form.
/predict	POST	Reads form values, creates a DataFrame, scales inputs, predicts flood risk, and renders the result pa
/home	GET	Redirects to `/`.

Prediction Flow in app.py

- Read each expected form value using the exact feature names.

- Validate that no submitted value is empty.

- Convert values to floats.

- Create a one-row pandas DataFrame with the training feature order.

- Apply the loaded StandardScaler.

- Predict using the loaded model.

- Render `chance.html` when prediction is 1; otherwise render `no_chance.html`.

Frontend Pages

The frontend uses clean HTML5 templates, a shared CSS file, and vanilla JavaScript validation. The prediction form submits by POST to `/predict`.

File	Purpose
templates/home.html	Landing page with project title, description, and Predict Flood button.
templates/index.html	Prediction form with inputs for all ten model features.
templates/chance.html	Red-themed flood-risk detected result page.
templates/no_chance.html	Green-themed no-risk result page.
static/css/main.css	Responsive styling, cards, colors, shadows, buttons, and layouts.
static/js/main.js	Client-side validation that prevents empty form submission.

Project Architecture

```
User Browser
    |
    v
Flask Routes in app.py
    |
    +-- GET /        -> templates/home.html
    +-- GET /predict -> templates/index.html
    +-- POST /predict
            |
            v
      Form Input Data
            |
            v
      pandas DataFrame with exact feature names
            |
            v
      models/transform.save (StandardScaler)
            |
            v
      models/floods.save (XGBClassifier)
            |
            v
      chance.html or no_chance.html
```

End-to-End Workflow

```
Dataset
  -> EDA
  -> Preprocessing
  -> Feature Scaling
  -> Decision Tree
  -> Random Forest
  -> KNN
  -> XGBoost
  -> Model Comparison
  -> Save Model
  -> Flask Deployment
  -> User Prediction
```

Installation Steps

- Clone the repository.

- Create a Python virtual environment with `python -m venv venv`.

- Activate the environment using `venv\Scripts\activate` on Windows or `source venv/bin/activate` on macOS/Linux.

- Install dependencies using `pip install -r requirements.txt`.

Dependencies in requirements.txt

The requirements file includes pandas, numpy, matplotlib, seaborn, scikit-learn, xgboost, Flask, joblib, openpyxl, and gunicorn with pinned versions.

Running the Project

- To regenerate the final model artifact, run `python src/train_models.py`.

- To start the Flask application locally, run `python app.py`.

- Open `http://127.0.0.1:5000/` in a browser.

The application requires both `models/floods.save` and `models/transform.save` to be present for prediction.

Deployment (Render)

The project README lists the Render deployment URL as `https://rising-waters-flood-prediction-nktt.onrender.com`. The requirements file includes `gunicorn`, which is commonly used to serve Flask applications in production-style deployments on platforms such as Render.

The README notes that the application is hosted on Render's free tier, so an idle service may take up to 60 seconds to wake up on the first request.

Results

The completed application supports end-to-end prediction through a web interface. For submitted weather and rainfall inputs, the backend returns one of two result pages: `Flood Risk Detected` or `No Flood Risk Detected`.

- The saved deployment model is an XGBoost classifier persisted as `models/floods.save`.

- The saved scaler is a StandardScaler persisted as `models/transform.save`.

- The dataset currently contains 99 no-risk records and 16 flood-risk records in the target column.

- Model evaluation metrics are printed by `src/train_models.py` during training rather than stored as a separate report file.

-

Screenshots

The repository contains four existing screenshot files in the `screenshots/` folder. This documentation references those files only and does not recreate or duplicate the images.

Screen	Existing File
Home Page	screenshots/home_page.png
Prediction Form	screenshots/prediction_form.png
Flood Result	screenshots/flood_result.png
No Flood Result	screenshots/no_flood_result.png

Challenges Faced

- Maintaining exact feature-name consistency between the dataset, training script, frontend form, and Flask prediction route.

- Persisting and reusing the same StandardScaler so that application inputs are transformed consistently with training data.

- Handling model selection fairly while ensuring XGBoost is selected when tied for best accuracy.

- Keeping the Flask backend simple while still loading model artifacts once at startup and handling prediction errors clearly.

- Building a responsive frontend without adding unnecessary backend complexity.

Conclusion

Rising Waters Flood Prediction System is a complete machine learning web application that demonstrates dataset exploration, preprocessing, feature scaling, model training, model comparison, model persistence, and Flask-based deployment. The current implementation uses a saved XGBoost classifier and StandardScaler to provide flood-risk predictions through a responsive frontend.

The project is suitable for internship and academic submission because it includes a clear ML workflow, organized project structure, reusable training code, persisted artifacts, and an operational Flask web interface.

References

- Project source files: `app.py`, `src/train_models.py`, `notebooks/FloodPrediction.ipynb`, frontend templates, and static assets.

- Dataset file: `data/flood_dataset.xlsx`.

- Model artifacts: `models/floods.save` and `models/transform.save`.

- Pandas documentation for data loading and DataFrame operations.

- Scikit-learn documentation for train-test split, StandardScaler, metrics, Decision Tree, Random Forest, and KNN.

- XGBoost documentation for XGBClassifier.

- Flask documentation for routing, templates, and request handling.

- Render documentation for Python web service deployment concepts.